Directed Containers in Trust Boundaries: How Proof Registries Are Comonads

Robin Langer

July 2026

Abstract

We describe how the *trust-boundaries* framework uses directed containers (Ahman–Chapman–Uustalu) as the organising mathematical structure for AI agent proof registries. A proof registry is a finitely-branching rose tree whose nodes carry trust labels; the directed container structure equips it with a comonad whose `extract` reads the root's trust level and whose `duplicate` replaces every node with the full sub-registry below it. The boundary rule—the core invariant of the trust system—is a coKleisli condition: a local predicate applied at every position produced by `duplicate`. Stability of this rule under `duplicate` (Theorem 3.3) makes cross-agent composition sound: a shared stub importing a validated sub-registry inherits a validated subtree with no re-checking required. The entire development, including the five directed container laws, the three comonad laws, coKleisli extension, and boundary stability, is formalised in sorry-free Lean 4.

Contents

1 Background: containers and directed containers **1**

2 Proof registries as directed containers **2**
2.1 The shape: finitely-branching rose trees 2
2.2 Trust labels and the labelled directed container 3
2.3 The comonad reading 3

3 The boundary rule as a coKleisli condition **3**

4 CoKleisli structure **4**

5 Directed containers as categories: cofunctors and composition **4**

6 Transport of trust and consolidation **5**

7 The Zappa–Szép product **5**

8 Summary: what directed containers buy us **5**

1 Background: containers and directed containers

We recall the definitions from Abbott–Altenkirch–Ghani and Ahman–Chapman–Uustalu, following the Lean formalisation in `lean/Basic.lean` and `lean/Directed.lean`.

Definition 1.1 (Container)**.** A *container* is a pair $C = (S, P)$ where S is a type of *shapes* and $P : S \to \text{Type}$ is a family of *positions*. Its *extension* is the polynomial functor

$$C(X) = \sum_{s:S} (P(s) \to X).$$

Container morphisms $(f, f^\sharp) : C \to D$ consist of a shape map $f : S_C \to S_D$ and a *contravariant* position map $f^\sharp_s : P_D(f\,s) \to P_C(s)$. The representation theorem (formalised in `Basic.lean`) establishes: container morphisms $C \to D$ are in bijection with natural transformations $C \Rightarrow D$.

Definition 1.2 (Directed container)**.** A *directed container* extends a container (S, P) with three operations:

- root : $\prod_{s:S} P(s)$ (the distinguished root position),
- $(-)\downarrow(-) : \prod_{s:S} P(s) \to S$ (the sub-shape at a position),
- $(-) \oplus (-) : \prod_{s:S} \prod_{p:P(s)} P(s\downarrow p) \to P(s)$ (the shift),

subject to five laws:

D1 $s\downarrow \mathrm{root}(s) = s$,
D2 $\mathrm{root}(s) \oplus q = q$ (transported along D1),
D3 $p \oplus \mathrm{root}(s\downarrow p) = p$,
D4 $s\downarrow(p \oplus q) = (s\downarrow p)\downarrow q$,
D5 $(p \oplus q) \oplus r = p \oplus (q \oplus r)$ (transported along D4).

The key insight of Ahman–Chapman–Uustalu: directed containers are exactly the containers whose extension functor carries a comonad structure. The comonad operations are:

$$\varepsilon\,\langle s, v\rangle = v(\mathrm{root}(s)),$$
$$\delta\,\langle s, v\rangle = \langle s,\ \lambda p.\,\langle s\downarrow p,\ \lambda q.\, v(p \oplus q)\rangle\rangle.$$

The three comonad laws follow from the five directed container laws: left counit from D1+D2, right counit from D3, coassociativity from D4+D5.

Definition 1.3 (Directed containers as categories)**.** A directed container D presents a small category $\mathcal{C}(D)$:

- objects are shapes $s : S$;
- a morphism $s \to s'$ is a position $p : P(s)$ with $s\downarrow p = s'$;
- the identity at s is $\mathrm{root}(s)$ (by D1);
- composition is the shift $\oplus$ (by D4).

The category laws are D2/D3 (identities) and D5 (associativity). Morphisms of directed containers are *cofunctors* of the presented categories (formalised in `DContCat.lean`, both round trips proved `rfl`).

2 Proof registries as directed containers

2.1 The shape: finitely-branching rose trees

A deployed proof registry (`proofs/registry/*.json`) is a finitely-branching tree. Each node represents a mathematical claim; its children are the sub-results the claim depends on.

Definition 2.1 (Rose tree)**.** The type `RTree` of finitely-branching rose trees is defined inductively:

$$\texttt{node}(n, \mathit{children}) \quad \text{where} \quad n : \mathbb{N}, \quad \mathit{children} : \mathrm{Fin}(n) \to \texttt{RTree}.$$

A leaf (dead end or ground check) is $\texttt{node}(0, !)$ where ! is the vacuous function from Fin(0).

Definition 2.2 (Position in a rose tree)**.** The type $\texttt{RPos}(t)$ of positions (paths from root to a node) in a tree t is:

- $\texttt{here} : \texttt{RPos}(t)$ (the root),
- $\texttt{child}(i, p) : \texttt{RPos}(\texttt{node}(n, c))$ where $i : \mathrm{Fin}(n)$ and $p : \texttt{RPos}(c(i))$.

The directed container structure on $(\texttt{RTree}, \texttt{RPos})$ is path manipulation:

Definition 2.3 (The `proofSearchN` directed container)**.**

$$\begin{aligned}
\mathrm{root}(t) &= \texttt{here},\\
t\downarrow\texttt{here} &= t,\\
t\downarrow\texttt{child}(i,p) &= c(i)\downarrow p,\\
\texttt{here}\oplus q &= q,\\
\texttt{child}(i,p)\oplus q &= \texttt{child}(i,\ c(i).\mathrm{shift}(p,q)).
\end{aligned}$$

Laws D1 and D2 hold *definitionally* (`rfl` in Lean). D3–D5 are proved by structural recursion on paths. This is formalised in `ProofSearchN.lean`.

2.2 Trust labels and the labelled directed container

In the deployed system, each node carries a *trust label*: a trust level from a totally ordered chain T, plus a Boolean dead-end flag.

Definition 2.4 (Labelled rose tree)**.** $\texttt{LTree}(T)$ extends `RTree` with a label $\ell = (\mathrm{trust}, \mathrm{dead})$ at every node:

$$\texttt{node}(\ell, n, \mathit{children}) \quad \text{where} \quad \ell : T\times\mathbb{B}, \quad n:\mathbb{N}, \quad \mathit{children} : \mathrm{Fin}(n)\to\texttt{LTree}(T).$$

The directed container $\texttt{trustRegistry}(T)$ has shapes $\texttt{LTree}(T)$ and positions $\texttt{LPos}(T)$. Labels play no role in navigation—they ride along as passengers—so the five laws are word-for-word those of `proofSearchN`. This is formalised in `TrustBoundary.lean`.

2.3 The comonad reading

The comonad induced by `trustRegistry` on $C(X) = \sum_{t:\texttt{LTree}}(\texttt{LPos}(t)\to X)$ has the following operational meaning:

Operation	Meaning for proof registries
$\varepsilon\,\langle t, v\rangle = v(\texttt{here})$	Read the trust label at the root: "is this conjecture proved?"
$\delta\,\langle t, v\rangle$	At every node, replace its label with the full sub-registry below it. Every claim now sees its entire dependency tree.
$\texttt{extend}\,f$	Apply a local analysis f (a coKleisli morphism) at every node, where f can inspect the whole sub-tree.

The validator's report modes—`report frontier`, `report dead-ends`, `report certificate-gap`—are coKleisli morphisms $W(\mathrm{Registry})\to\mathrm{Report}$: strategies that inspect the entire tree before summarising.

3 The boundary rule as a coKleisli condition

Definition 3.1 (Local boundary rule)**.** For a threshold $\theta\in T$ (deployed: $\theta = \texttt{proved}$), the *local boundary rule* at a node $\texttt{node}(\ell, n, c)$ is:

$$\theta\le\ell.\mathrm{trust} \implies \forall\, i:\mathrm{Fin}(n),\ c(i).\mathrm{dead} = \texttt{true} \ \vee\ \theta\le c(i).\mathrm{trust}.$$

"If this node claims θ or above, every non-dead child must claim θ or above." Dead ends are exempt: abandoned approaches are not required premises.

Definition 3.2 (Global boundary rule)**.** The *global boundary rule* $\mathrm{Ok}(\theta, t)$ holds when the local check passes at *every* position:

$$\mathrm{Ok}(\theta, t) \iff \forall\, p : \mathtt{LPos}(t),\ \mathrm{localOk}(\theta,\ t{\downarrow}p).$$

This is the coKleisli form: apply the local predicate to every sub-search produced by δ (duplicate).

Theorem 3.3 (Duplicate-stability of the boundary rule)**.** *If* $\mathrm{Ok}(\theta, t)$ *then* $\mathrm{Ok}(\theta, t{\downarrow}p)$ *for every position* p*.*

Proof. The proof is a one-line transport along D4. Given $q : \mathtt{LPos}(t{\downarrow}p)$, we need $\mathrm{localOk}(\theta, (t{\downarrow}p){\downarrow}q)$. By D4, $(t{\downarrow}p){\downarrow}q = t{\downarrow}(p \oplus q)$, so the hypothesis $\mathrm{Ok}(\theta, t)$ applied at position $p \oplus q$ gives exactly the required local check. In Lean:

```
theorem ok_sub ( : T) (t : LTree T) (h : Ok  t) (p : LPos t) :
    Ok  (t.sub p) :=
  fun q => t.sub_shift p q  h (t.shift p q)
```

□

Remark 3.4. Theorem 3.3 is the load-bearing fact for cross-agent composition. When Clio imports a sub-registry from Rick via a shared stub, the stub points into a validated registry. By Theorem 3.3, the subtree rooted at that stub is *automatically* validated—no re-checking is required across the agent boundary.

4 CoKleisli structure

The directed container comonad has a full coKleisli category, formalised in `CoKleisli.lean`.

Definition 4.1 (CoKleisli extension)**.** Given $f : W(X) \to Y$, the extension is:

$$\mathrm{extend}(f)\,\langle s, v\rangle = \langle s,\ \lambda p.\, f\,\langle s{\downarrow}p,\ \lambda q.\, v(p \oplus q)\rangle\rangle.$$

This equals $W(f) \circ \delta$ (map followed by comultiplication).

The coKleisli category has:

- **Objects**: types.
- **Morphisms** $X \to Y$: functions $W(X) \to Y$ (strategies that see the whole tree before producing a result).
- **Composition**: $g \bullet f = g \circ \mathrm{extend}(f)$.
- **Identity**: ε (extract the root).

The three coKleisli laws are proved:

1. $\mathrm{extend}(\varepsilon) = \mathrm{id}$ (from D3),
2. $\varepsilon \circ \mathrm{extend}(f) = f$ (from D1+D2),
3. $\mathrm{extend}(g \circ \mathrm{extend}(f)) = \mathrm{extend}(g) \circ \mathrm{extend}(f)$ (from D4+D5).

Definition 4.2 (Canonical strength)**.** The strength $\sigma : W(A \times B) \to A \times W(B)$ is:

$$\sigma\,\langle s, v\rangle = \big(\pi_1(v(\mathrm{root}(s))),\ \langle s,\ \pi_2 \circ v\rangle\big).$$

This factors a uniform context A out of a labelled tree. In the Python validator (`trustcheck.py`), this is `dc_strength`: it separates metadata from the trust-labelled content.

5 Directed containers as categories: cofunctors and composition

The equivalence **DCont** $\simeq$ **Cat** of Ahman–Uustalu (2016) is formalised in `DContCat.lean`. For trust boundaries, the key consequence is:

Theorem 5.1 (Morphisms are cofunctors)**.** *A directed container morphism* $(f, f^\sharp) : C \to D$ *(shape map covariant, position map contravariant, satisfying M0–M2) is exactly a cofunctor* $\mathcal{C}(C) \to \mathcal{C}(D)$*. The bijection has both round trips* `rfl`*.*

This matters for trust composition: cross-agent shared stubs are container morphisms between registries. The trust cap (a stub's local trust must not exceed the canonical trust) is the morphism condition. Cofunctor composition gives transitive cross-agent citation: if Clio cites Rick and Rick cites MacBeth, the composite cofunctor gives the end-to-end trust transport. The Lean proof (`Cofunctor.comp`) verifies that cofunctor composition is strictly associative and unital—both round trips are `rfl`.

6 Transport of trust and consolidation

The trust system theory (`Trust.lean`) connects to directed containers through two results:

Lemma 6.1 (Transport of trust)**.** *Given a morphism of trust systems* $(F, \varphi) : (D, T, s) \to (D', T', s')$ *where* $\varphi : T \to T'$ *is monotone and strict at* $\bot$*, the canonical trust transports laxly:*

$$\varphi(\tau(x)) \leq \tau'(F(x)).$$

This is the mathematical guarantee behind the φ maps in deployment descriptors. When Clio imports a claim from Rick, the interface map φ translates trust levels:

Rick's `checked-sober`	$\xrightarrow{\varphi}$	Clio's `computed`
Rick's `proved`	$\xrightarrow{\varphi}$	Clio's `proved`
Rick's `hunch`	$\xrightarrow{\varphi}$	Clio's `speculative`

The transport lemma guarantees that φ never inflates trust: Clio cannot claim more than φ of Rick's canonical trust.

Theorem 6.2 (Consolidation preservation)**.** *A* consolidation *is a morphism of trust systems at* $\varphi = \mathrm{id}$ *together with a backward position map (every checking step in the target pulls back to a checking step in the source, premises corresponding, cached grade* $\leq$ *source grade). Under consolidation, canonical trust is preserved on the nose:* $\tau'(F(x)) = \tau(x)$*.*

The position map is the container-theoretic content: shapes (checking steps) map forward, positions (justifications) map backward. This is precisely the data of a container morphism—covariant on shapes, contravariant on positions. The consolidation theorem says: when both halves are present (not just shapes), trust is preserved exactly.

Consolidations compose (`Consolidation.comp`): the pipeline scratch $\to$ synthesis $\to$ proof preserves τ end-to-end iff it preserves τ layer by layer. The composition is cofunctor composition on the position maps—contravariantly, as it must be.

7 The Zappa–Szép product

The formalisation includes a Zappa–Szép product of monoids (`ZappaSzep.lean`), proving that the four Brin cocycle axioms ZS1–ZS4 are equivalent to associativity of the product. This connects to the broader programme: directed containers model the *internal* structure of each agent's proof registry (the comonad), while Zappa–Szép products model the *interaction* between agents whose symmetry groups act on each other. The cocycle conditions are the trust-composition analogue of the directed container laws: both encode coherence of a layered algebraic structure.

8 Summary: what directed containers buy us

1. **The comonad is the validator's logic.** Extract reads trust at the root; duplicate gives every node its full dependency context; the boundary rule is a coKleisli condition.

2. **Stability under duplicate gives compositional validation.** Theorem 3.3: validated registries have validated sub-registries. Shared stubs inherit validity for free.

3. **CoKleisli morphisms are report modes.** Each `trustcheck.py report` subcommand is a coKleisli morphism: it inspects the whole sub-tree at every position before producing a summary.

4. **Container morphisms are cross-agent links.** Shared stubs between agents are container morphisms (shapes forward, positions backward). The trust cap is the morphism condition.

5. **Cofunctors compose transitively.** Multi-agent citation chains (Clio $\to$ Rick $\to$ MacBeth) compose as cofunctors. Trust transport along the composite is guaranteed by the Lean-verified `tau_transport`.

6. **Consolidation = container morphism at $\varphi = \mathrm{id}$ with position map.** This distinguishes genuine consolidation (both halves present, trust preserved) from shape-only restatement (positions lost, trust degrades to $\bot$).

7. **Everything is sorry-free in Lean 4.** The five directed container laws, three comonad laws, coKleisli extension laws, boundary stability, trust transport, consolidation preservation, the circular-citation counterexample, and the shape-only degradation counterexample—all proved with no sorry, no Mathlib dependency.

Acknowledgements

The directed container framework is due to Ahman, Chapman, and Uustalu. The equivalence **DCont** $\simeq$ **Cat** and the cofunctor characterisation of morphisms are from Ahman and Uustalu (2016). The application to trust systems emerged from work with Neil Ghani at Kodamai.